

Spezifikation der Eingabevalidierung

Portfolio-Aufgabe „Todo-Liste“ – Programmierung, Sommersemester 2026 Dozent: Dr. Thomas Sonntag

1. Ziel der Eingabevalidierung

Das Programm liest in einer Schleife Befehle von der Konsole. Diese Eingaben können fehlerhaft sein. Ziel der Validierung ist:

1. Jede ungültige Eingabe wird erkannt, bevor sie den Datenbestand verändert.
2. Bei einer ungültigen Eingabe gibt das Programm eine kurze, verständliche Fehlermeldung aus.
3. Bei Tippfehlern wird, wenn möglich, der vermutete Befehl vorgeschlagen.
4. Das Programm bricht nicht ab, sondern liest sofort die nächste Eingabe.
5. Das Programm endet ausschließlich auf den Befehl `quit` (oder das Schließen von stdin).

2. Gültige Befehle

Es existieren sechs Befehle. Jede andere Eingabe ist ungültig. `help` ist eine reine UX-Hilfe ohne State-Veränderung.

Befehl	Argument	Wirkung
<code>add <description></code>	Beschreibung	Neues Todo im Status <code>open</code> anlegen
<code>list</code>	keines	Alle Todos als Tabelle anzeigen
<code>done <id></code>	positive ganze Zahl	Status des Todos auf <code>done</code> setzen
<code>delete <id></code>	positive ganze Zahl	Todo aus der Liste entfernen
<code>help</code>	keines	Übersicht aller Befehle anzeigen
<code>quit</code>	keines	Programm beenden

Befehlsnamen werden case-sensitive verglichen (nur Kleinbuchstaben). Beschreibungen dürfen Groß- und Kleinbuchstaben, Umlaute und gängige Satzzeichen enthalten.

3. Allgemeine Validierungsregeln

Diese Regeln gelten für jede Eingabe, unabhängig vom Befehl:

1. Eine leere Eingabe oder eine Eingabe nur aus Leerzeichen ist ungültig.
2. Der erste Token muss exakt einer der sechs gültigen Befehle sein.
3. Bei unbekanntem Befehl wird, sofern eine ähnliche Schreibweise eines gültigen Befehls existiert, ein Vorschlag in der Fehlermeldung ergänzt (z. B. `Meintest du: add?`).
4. Bei Befehlen ohne Argument (`list`, `help`, `quit`) darf kein zusätzlicher Text folgen.
5. Bei Befehlen mit Argument (`add`, `done`, `delete`) muss das Argument vorhanden und gültig sein.
6. Ungültige Eingaben verändern den Zustand der Todo-Liste nicht.

4. Validierung je Befehl

add `<description>`

- Die Beschreibung muss vorhanden sein.
- Whitespace am Anfang und Ende wird abgeschnitten.
- Nach dem Abschneiden darf die Beschreibung nicht leer sein.
- Die Beschreibung darf höchstens **200 Zeichen** lang sein.
- Die Beschreibung darf keine Steuerzeichen enthalten (Codepoints < 32, z. B. `\n`, `\t`, `\r`).
- Umlaute und gängige Satzzeichen (`?!,:;äöüÄÖÜß`) sind erlaubt.
- Die zugewiesene ID wird automatisch vergeben (siehe Abschnitt 7).

list

- Es darf kein Argument folgen.
- Bei leerer Liste wird genau die Meldung `keine Aufgaben vorhanden` ausgegeben.
- Sonst wird eine ASCII-Tabelle mit Header `ID | Beschreibung | Status` und einer Zeile je Todo ausgegeben. Die Spaltenbreite passt sich an den längsten Inhalt an.

done <id>

- Genau ein Argument: eine ID.
- Die ID muss eine positive ganze Zahl sein (> 0, ohne Vorzeichen, ohne Dezimalpunkt).
- Die ID muss zu einem existierenden Todo gehören.
- Mehrfaches `done` auf dasselbe Todo ist erlaubt und ändert den Status nicht.

delete <id>

- Genau ein Argument: eine ID.
- Die ID muss eine positive ganze Zahl sein.
- Die ID muss zu einem existierenden Todo gehören.
- Gelöschte IDs werden **nicht** wiederverwendet.

help

- Es darf kein Argument folgen.
- Wirkung: Anzeige der Befehlsübersicht. Verändert den Zustand nicht.

quit

- Es darf kein Argument folgen.
- Wirkung: kontrollierter Programmstop mit der Ausgabe `Bye!`.

5. Verhalten bei Fehlern

1. Bei jeder Verletzung der Regeln aus den Abschnitten 3 oder 4 wird intern eine `ValidationException` ausgelöst.
2. Die Hauptschleife fängt diese Exception ab und schreibt ihre Nachricht auf die Konsole.
3. Anschließend liest die Schleife sofort die nächste Eingabe – das Programm läuft weiter.
4. Bei `quit` wird eine `QuitException` ausgelöst; die Hauptschleife beendet das Programm regulär.

6. Beispiele für falsche Eingaben

Eingabe	Reaktion (Konsolenausgabe)
(leer)	Fehler: Leere Eingabe.
	Fehler: Leere Eingabe.
ad Milch	Fehler: Unbekannter Befehl 'ad'. Meintest du: add? Tippe 'help' fuer eine Uebersicht.
don 1	Fehler: Unbekannter Befehl 'don'. Meintest du: done? Tippe 'help' fuer eine Uebersicht.
delet 1	Fehler: Unbekannter Befehl 'delet'. Meintest du: delete? Tippe 'help' fuer eine Uebersicht.
xyzzz	Fehler: Unbekannter Befehl 'xyzzz'. Tippe 'help' fuer eine Uebersicht.
Add Milch	Fehler: Unbekannter Befehl 'Add'. Meintest du: add? Tippe 'help' fuer eine Uebersicht.
add	Fehler: Beschreibung darf nicht leer sein.
add	Fehler: Beschreibung darf nicht leer sein.
add xxxxxx... (>200)	Fehler: Beschreibung darf hoechstens 200 Zeichen lang sein (war N).
add abc<TAB>def	Fehler: Beschreibung darf keine Steuerzeichen (z.B. Tab, Zeilenumbruch) enthalten.
list extra	Fehler: Der Befehl list erwartet kein Argument.
done	Fehler: Der Befehl done erwartet eine ID als Argument.
done abc	Fehler: ID muss eine positive ganze Zahl sein.
done -1	Fehler: ID muss eine positive ganze Zahl sein.
done 0	Fehler: ID muss eine positive ganze Zahl sein.
done 1.5	Fehler: ID muss eine positive ganze Zahl sein.
done 99	Fehler: Aufgabe mit ID 99 wurde nicht gefunden.
delete	Fehler: Der Befehl delete erwartet eine ID als Argument.
delete x	Fehler: ID muss eine positive ganze Zahl sein.
delete 99	Fehler: Aufgabe mit ID 99 wurde nicht gefunden.
help me	Fehler: Der Befehl help erwartet kein Argument.
quit now	Fehler: Der Befehl quit erwartet kein Argument.

Nach jeder dieser Reaktionen wartet das Programm auf die nächste Eingabe.

7. ID-Vergabe und Eindeutigkeit

- IDs starten bei 1.
- Bei jedem `add` wird die aktuell vorgesehene `next_id` als neue ID verwendet und anschließend um 1 erhöht.
- Ein `delete` verringert `next_id` nicht und verschiebt keine existierenden IDs.
- Damit bleiben IDs für die gesamte Programmlaufzeit eindeutig, auch nach `delete`.

Beispiel:

```
add A    -> ID 1, next_id = 2
add B    -> ID 2, next_id = 3
delete 2
add C    -> ID 3, next_id = 4
```

8. Vorschläge bei Tippfehlern

Wenn der erste Token kein gültiger Befehl ist, prüft das Programm mit `difflib.get_close_matches` (Python-Standardbibliothek, kein externes Framework), ob es einen ähnlichen Befehl gibt. Cutoff = 0.6. Bei einem Treffer wird der Vorschlag in der Fehlermeldung ergänzt. Andernfalls wird nur auf `help` hingewiesen.

9. Automatisches Testen von Input und Output

Die Logik ist bewusst in einen Functional Core (`todo.py`) und eine kleine Imperative Shell (`main.py`, `run_todo_app`, `get_input_from_user`) aufgeteilt. Dadurch sind verschiedene Teststrategien möglich:

- **Reine Logik-Tests:** `parse_command`, `to_id`, `validate_description`, `suggest_command`, `add_todo`, `mark_todo_done`, `delete_todo`, `format_todos`, `format_help`, `execute_command` werden direkt mit `pytest` und `pytest.raises` gegen erwartete Werte und Exceptions geprüft. Keine Konsole nötig.
- **Mocking von `input()`:** `unittest.mock.patch("builtins.input", return_value=...)` oder `pytest's monkeypatch.setattr("builtins.input", ...)` ersetzen `input()` während eines Tests.
- **Capturing von `print()`:** `pytest` stellt die Fixture `capsys` bereit. Damit liest ein Test den auf `stdout` geschriebenen Text aus und prüft ihn gegen die Spezifikation.
- **Schleifentest:** Eine Sequenz von Eingaben kann als Liste an `mock.side_effect` oder über `monkeypatch` als Iterator übergeben werden. Mit `pytest.raises(QuitException)` oder einem abschließenden `quit` lässt sich `run_todo_app` Ende-zu-Ende testen.

Die in `src/todo_test.py` enthaltenen 81 Tests decken Validierung, Befehlsparser, Vorschläge, Todo-Operationen, Beschreibungs-Limits, Tabellenformatierung, Hilfetext, Fehler- und Randfälle sowie Input-Mocking ab. Die Schleifentest-Strategie ist zusätzlich als Code umgesetzt: drei End-zu-End-Tests simulieren `run_todo_app` mit `monkeypatch` (Eingabesequenz als Iterator) und prüfen die Konsolenausgabe über `capsys` - inklusive Fehlererholung und Strg+D (EOF).